B12 Model evaluation

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B12 Model evaluation 3
B12-L1 Choose the metric from the decision 3
B12-L2 Calibration, when the probability itself matters 7
B12-L3 Imbalanced classes and error analysis by segment 10
Track B best-practice checklist 15
Appendix: solutions and answer keys 16

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B12 Model evaluation

Roadmap: Prerequisite for all months; Months 4 to 6: CNNs; Months 10 to 12: Transformers applied. Metric choice, calibration, imbalanced classes and error analysis.

B12-L1 Choose the metric from the decision

Concept

A metric is a summary of the mistakes you care about. Choose it from the decision the model supports, before you train anything.

Accuracy fails on imbalanced data. A test set from this course's build has 2.4% positives. Predicting "negative" for everyone scores accuracy **0.976**. A logistic regression at the default 0.5 threshold scored 0.9755, with **recall 0.0**: it found no positives at all.

Better summaries, all in scikit-learn 1.8:

- **ROC AUC** (`roc_auc_score`): the chance a random positive ranks above a random negative. It ignores the threshold and the class balance. Good for comparing rankers. It hides how many false positives come with each true positive when positives are rare.
- **Average precision**, the area under the precision-recall curve (`average_precision_score`): sensitive to rare positives. A random ranking scores the **prevalence**, 0.025 here. The same logistic regression scored **0.30**, twelve times the baseline. Always quote the baseline next to it.
- **Precision and recall at a threshold**: precision is the share of flagged cases that are real. Recall is the share of real cases that are flagged. These map straight onto work: "of 100 customers we call, 30 were going to churn" and "we reach 64% of the churners".
- **Cost-weighted metrics**: if a missed fraud costs £400 and a false alarm costs £5 of analyst time, compute the total cost at each threshold and pick the cheapest.

Pick the **threshold** from the decision too. 0.5 is a default of the library, not a business rule. If the team can review 200 cases a day, the threshold is whatever flags 200 cases.

Report the metric with the prevalence and the number of positives in the test set. A precision of 0.30 from 12 positives is not the same evidence as from 1,200.

Why it matters

A claims triage model was reported as "97.6% accurate". It flagged nothing. Fraud prevalence was 2.4%, so a model that approved every claim scored the same. The number went into a board paper before anyone asked how many frauds it caught. The answer was zero.

Worked example: weak code

```
pred = model.predict(X_test)
print("accuracy", accuracy_score(y_test, pred))
```

1. **Line 1.** predict() uses a 0.5 threshold that nobody chose.
2. **Line 2.** Accuracy on rare positives rewards predicting the majority class.

Worked example: strong code

```
import numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score


def evaluate(y_true: np.ndarray, scores: np.ndarray, threshold: float) -> dict[str, float | None]:
    """Ranking and threshold metrics, with the baseline to compare against."""
    flagged = scores >= threshold
    tp = int((flagged & (y_true == 1)).sum())
    n_flagged, n_pos = int(flagged.sum()), int((y_true == 1).sum())
    return {
        "prevalence": float(y_true.mean()),
        "roc_auc": float(roc_auc_score(y_true, scores)),
        "average_precision": float(average_precision_score(y_true, scores)),
        "precision": tp / n_flagged if n_flagged > 0 else None,
        "recall": tp / n_pos if n_pos > 0 else None,
    }
```

1. **Line 11.** The baseline for average precision, reported alongside it.
2. **Line 14.** No flags, no precision: None, not a division error or a misleading 0.

Choose metrics and thresholds from the decision. Report prevalence and counts beside every number.

Recap card

- **Accuracy at 2.4% prevalence** 0.976 for predicting all negatives. Useless as a headline.
- **Baseline for average precision** The prevalence. Report it beside the score.
- **Where the threshold comes from** The decision: review capacity or costs. Not the 0.5 default.
- **Precision and recall** Precision: share of flags that are real. Recall: share of real cases flagged.

Practice

Exercise 1. Multiple choice (level 1)

Positives are 2.4% of the test set. A model has accuracy 0.976. What do you know?

- a) The model is excellent.
- b) Nothing useful yet. Predicting all negatives scores the same.
- c) The model finds 97.6% of positives.
- d) The model is calibrated.

Why?

- r1) Accuracy is dominated by the 97.6% majority class.
- r2) Accuracy is never useful.

Exercise 2. Predict the output (level 2)

What does this print, to two decimals? (scikit-learn 1.8, NumPy 2.4)

```
import numpy as np
from sklearn.metrics import average_precision_score

rng = np.random.default_rng(0)
y = (rng.random(20000) < 0.05).astype(int)
random_scores = rng.random(20000)
print(round(average_precision_score(y, random_scores), 2))
```

Exercise 3. Fill the gap (level 2)

Flag the top 200 cases a day, whatever their scores.

```
threshold = np.sort(scores)[_____(1)]
flagged = scores >= _____(2)
```

Exercise 4. Write the function (level 3)

Write `evaluate(y_true, scores, threshold) -> dict[str, float | None]` returning:

- `prevalence`: mean of `y_true`.
- `roc_auc` and `average_precision` from scikit-learn.
- `precision` and `recall` at `scores >= threshold`, computed from counts. Return `None` for precision when nothing is flagged, and for recall when there are no positives.

Your code must pass these checks:

- Check prevalence, ROC AUC, and precision and recall from counts at scores >= threshold.
- average_precision must come from average_precision_score(y_true, scores).
- No flagged cases: precision is None, recall is 0.0.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Ranking metrics, threshold metrics and the prevalence baseline.
def evaluate(
    y_true: np.ndarray,
    scores: np.ndarray,
    threshold: float,
) -> dict[str, float | None]: ...
```

Starter code:

```python
from sklearn.metrics import accuracy_score


def evaluate(y_true, scores, threshold):
    return {"accuracy": accuracy_score(y_true, scores >= threshold)}
```

Exercise 5. Architecture design (level 4)

Choose the headline metric for each model.

Options: Precision among the top k flagged, Accuracy, ROC AUC, Total cost at the chosen threshold.

1. Fraud: 1% prevalence, analysts review 150 cases a day
2. Compare two ranking models before any threshold exists
3. Loan approval: a default costs £4,000, a wrongly refused customer costs £300 of margin

Sources

- On make_classification data with 2.4% positives, all-negative accuracy was 0.976, logistic regression at 0.5 had recall 0.0, average precision 0.30 against a random 0.025. (verified: Run with scikit-learn 1.8.0 and 1.9.1 in this repository (random_state=0).)

B12-L2 Calibration, when the probability itself matters

Concept

A model is **calibrated** when its probabilities mean what they say: of all customers scored 0.30, about 30% churn. Ranking metrics such as ROC AUC ignore calibration. A model can rank perfectly and still say 0.9 where the truth is 0.1.

Calibration matters whenever someone uses the number itself: expected revenue (`probability × value`), a forecast count of churners, a risk shown to a person, or thresholds agreed in probability terms.

Check it with a calibration table: group predictions into bins, then compare the mean predicted probability with the observed rate in each bin. scikit-learn 1.8 gives the same with `calibration_curve(y, p, n_bins=10)`. Plot observed against predicted. A calibrated model follows the diagonal.

Summarise with the **Brier score** (`brier_score_loss`), the mean squared error of the probabilities. Compare it with a constant model that always predicts the prevalence.

A common cause of bad calibration is **reweighting for imbalance**. In this course's build, a logistic regression on data with 2.4% positives had a mean predicted probability of 0.024 and a Brier score of 0.021. The same model with `class_weight="balanced"` had a mean predicted probability of **0.42**, and a Brier score of **0.199**, worse than predicting 0.024 for everyone (0.024). Its AUC was slightly higher. Its probabilities were useless as probabilities.

Fix it with `CalibratedClassifierCV(model, method="sigmoid", cv=5)`, which fits a calibration map on held-out folds. Use `method="isotonic"` only with plenty of data, roughly a thousand positives or more, because it overfits small samples. Check calibration again on the test set after fitting.

If you only need a ranking or a top-k list, calibration does not matter. Say which one you deliver.

Why it matters

A retention team estimated campaign value as the sum of churn probabilities times customer margin. The model had been trained with balanced class weights. Its average churn probability was 0.41 on a base that churned at 3% a month. The business case promised ten times the saves the campaign could deliver, and budget was approved on it.

Worked example: weak code

```
model = LogisticRegression(class_weight="balanced").fit(X_train, y_train)
expected_churners = model.predict_proba(X_live)[:, 1].sum()
```

1. **Line 1.** Balanced weights push probabilities towards 0.5.
2. **Line 2.** Sums miscalibrated probabilities as if they were real rates.

Worked example: strong code

```
import numpy as np
import pandas as pd


def calibration_table(y: np.ndarray, p: np.ndarray, n_bins: int) -> pd.DataFrame:
    """Mean predicted and observed rate per equal-width bin. Empty bins are left out."""
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    idx = np.searchsorted(edges[1:-1], p, side="right")
    df = pd.DataFrame({"bin": idx, "y": y, "p": p})
    table = df.groupby("bin").agg(mean_predicted=("p", "mean"), observed=("y", "mean"), n=("y", "size"))
    return table.reset_index(drop=True)
```

1. **Line 8.** Assigns each probability to an equal-width bin.
2. **Line 10.** Predicted against observed, with the count behind each row.

When the probability is used as a number, check calibration, compare the Brier score with a constant baseline, and recalibrate on held-out data.

Recap card

- **Calibrated means** Of cases scored p, about p are positive.
- **Check calibration** Calibration table or calibration_curve, plus Brier score against the constant prevalence baseline.
- **class_weight="balanced" and probabilities** Pushes them towards 0.5. Mean p went from 0.024 to 0.42 at 2.4% prevalence.
- **Recalibrate** CalibratedClassifierCV(model, method="sigmoid", cv=5). Isotonic only with plenty of data.

Practice

Exercise 1. Multiple choice (level 1)

A model has ROC AUC 0.85. Of customers it scores between 0.8 and 0.9, 20% churn. What is true?

- a) It ranks well but is poorly calibrated.
- b) It is well calibrated.
- c) The AUC must be wrong.
- d) It ranks badly.

Why?

- r1) AUC measures ordering only. The probabilities can be far off while the order is right.
- r2) Any model with AUC above 0.8 is calibrated.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
from sklearn.metrics import brier_score_loss

y = [0, 0, 0, 1]
print(brier_score_loss(y, [0.25, 0.25, 0.25, 0.25]), brier_score_loss(y,
    [0.5, 0.5, 0.5, 0.5]))
```

Exercise 3. Fill the gap (level 2)

Recalibrate a classifier with a sigmoid map fitted on held-out folds.

```
calibrated = _____(1)(model, method="_____(2)", cv=5).fit(X_train,
    y_train)
```

Exercise 4. Write the function (level 3)

Write `calibration_table(y, p, n_bins) -> pd.DataFrame`. Split [0, 1] into `n_bins` equal-width bins. Assign each probability with `np.searchsorted(edges[1:-1], p, side="right")`. Return one row per **non-empty** bin, in bin order, with columns `mean_predicted`, `observed` (mean of `y`) and `n`, and a default 0..k index.

Your code must pass these checks:

- Each bin needs its mean predicted probability, observed rate and count.
- Leave out empty bins, and reset the index.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Mean predicted probability and observed rate per equal-width bin.
def calibration_table(
    y: np.ndarray,
    p: np.ndarray,
    n_bins: int,
) -> pd.DataFrame: ...
```

Starter code:

```python
import numpy as np
import pandas as pd


def calibration_table(y, p, n_bins):
    return pd.DataFrame({"mean_predicted": [p.mean()], "observed":
    [y.mean()]})
```

Exercise 5. Architecture design (level 4)

A churn model will be used two ways. Decide what each use needs.

Options: Good ranking only, Calibrated probabilities.

1. Pick the top 500 customers to call each week
2. Forecast the number of churners next month for finance
3. Class imbalance handling for the forecast model

Sources

- Logistic regression at 2.4% prevalence: mean p 0.024 and Brier 0.021; with class_weight="balanced", mean p 0.42 and Brier 0.199; constant baseline 0.024. (verified: Run with scikit-learn 1.8.0 and 1.9.1 in this repository (make_classification, random_state=0).)
- The table matches sklearn.calibration.calibration_curve with strategy="uniform" on test data. (verified: Compared with np.allclose on scikit-learn 1.8.0 in this repository.)

B12-L3 Imbalanced classes and error analysis by segment

Concept

Imbalance. When positives are rare, a model trained as usual predicts low probabilities, and the default 0.5 threshold flags almost nothing. That is a threshold problem, not always a model problem.

In this course's build (2.4% positives, logistic regression, scikit-learn 1.8), the plain model at 0.5 flagged 3 cases and found none. Lowering the threshold to

0.1 flagged 284 cases with precision 0.36 and recall 0.42. At 0.02 it reached recall 0.63 with precision 0.04. The same model trained with `class_weight="balanced"` reached recall 0.64 and precision 0.05 at 0.5: a very similar trade-off, with probabilities that were no longer calibrated (lesson B12-L2).

Options, in the order to try them:

1. **Train normally and choose the threshold** from capacity or cost (lesson B12-L1). Probabilities stay calibrated.
2. **Class weights** when the model struggles to learn the minority class at all. Recalibrate afterwards if you need probabilities.
3. **Resampling** (over- or under-sampling) **inside the training folds only**. Never resample the test set: its class balance must match production.

Error analysis. One overall metric hides where a model fails. Break the test results down by segment: region, device, customer tenure, survey mode. For each segment report the count, positives, precision and recall. A model with recall 0.60 overall can have recall 0.15 for new customers, the group the campaign most wants to reach.

Then read examples. Take the 20 most confident false positives and the 20 most confident false negatives, and look at them with someone who knows the domain. Patterns in those 40 rows often point to a missing feature, a label error or a leak.

Report segment results with their counts. A recall of 0.20 from 5 positives is a question, not a finding.

Why it matters

A complaint-routing model reported recall 0.71 overall. Complaints arriving by the new web chat were 18% of the volume and had recall 0.22: the model had been trained mostly on email text, and chat messages were shorter and informal. Urgent chat complaints waited in the general queue for three weeks before a supervisor noticed. A per-channel breakdown at validation would have shown it.

Worked example: weak code

```
model = LogisticRegression().fit(X_train, y_train)
print("recall", recall_score(y_test, model.predict(X_test)))
```

1. **Line 2.** The 0.5 default flags almost nothing on rare positives, and one overall number hides weak segments.

Worked example: strong code

```python
import pandas as pd


def metrics_by_segment(df: pd.DataFrame, segment: str) -> pd.DataFrame:
    """Count, positives, precision and recall per segment. Columns y and flagged are 0/1."""
    rows = []
    for name, g in df.groupby(segment, sort=True):
        tp = int(((g["y"] == 1) & (g["flagged"] == 1)).sum())
        n_flag, n_pos = int(g["flagged"].sum()), int(g["y"].sum())
        rows.append({
            segment: name,
            "n": len(g),
            "positives": n_pos,
            "precision": tp / n_flag if n_flag > 0 else None,
            "recall": tp / n_pos if n_pos > 0 else None,
        })
    return pd.DataFrame(rows)
```

1. **Line 7.** One row per segment, in a stable order.
2. **Line 13.** Counts beside every rate, so small segments are read with care.
3. **Line 15.** No positives, no recall: None instead of a crash or a fake 0.

Treat rare positives with a chosen threshold first, then break results down by segment and read the worst errors.

Recap card

- **First response to rare positives** Train normally and choose the threshold from capacity or cost.
- **Resampling rule** Only inside training folds. Never the test set.
- **Error analysis** Metrics per segment with counts, then read the most confident errors.
- **Pooled vs segment recall** Pooling weights large segments. Overall 0.7 can hide a segment at 0.2.

Practice

Exercise 1. Multiple choice (level 1)

A model trained on 2% positives flags nothing at the default threshold. What should you try first?

- a) Oversample the test set to 50% positives.
- b) Lower the threshold to match review capacity or costs.
- c) Switch to a deep neural network.

- d) Report accuracy instead.

Why?

- r1) The model's ranking may be fine. Only the cut-off is wrong for rare positives.
- r2) Lower thresholds always improve models.

Exercise 2. Predict the output (level 2)

What does this print?

```
segments = {"email": (80, 100), "chat": (4, 20)}
for name, (found, positives) in segments.items():
    print(name, found / positives)
print("overall", sum(f for f, _ in segments.values()) / sum(p for _, p in
    segments.values()))
```

Exercise 3. Put in order (level 2)

Order the steps for handling a rare positive class.

- Read the most confident false positives and false negatives
- Break test results down by segment, with counts
- Train the model normally
- Choose the threshold from capacity or cost on validation data
- Split train and test, stratified, test left at its natural balance

Exercise 4. Write the function (level 3)

Write `metrics_by_segment(df, segment) -> pd.DataFrame`. `df` has 0/1 columns `y` and `flagged` and a segment column named by `segment`. Return one row per segment, sorted by segment, with columns `segment` (named as the input column), `n`, `positives`, `precision` and `recall`. Use `None` for precision when a segment has no flags, and for recall when it has no positives.

Your code must pass these checks:

- Return the columns in the order given, one row per segment, sorted by segment.
- Count true positives per segment: precision = tp / flagged, recall = tp / positives.
- No flags gives no precision, and no positives gives no recall: missing, not 0.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Count, positives, precision and recall per segment.
```

```
def metrics_by_segment(
    df: pd.DataFrame,
    segment: str,
) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def metrics_by_segment(df, segment):
    return df.groupby(segment)["y"].mean().reset_index()
```

Exercise 5. Spot the bug (level 4)

Validation reports recall 0.78, but production recall is 0.41. Tap the line that makes the evaluation unrepresentative, and fix the function.

What you need to know.

`df.sample(frac=5.0, replace=True, random_state=seed)` draws rows at random, with replacement, five times as many as the frame holds. Each row appears about five times. `pd.concat([a, b])` stacks two frames, rows of `a` first.

```
import pandas as pd


def prepare_sets(train: pd.DataFrame, test: pd.DataFrame, seed: int) ->
   tuple[pd.DataFrame, pd.DataFrame]:
    """Oversample positives so the model sees more of them."""
    pos_train = train[train["y"] == 1].sample(frac=5.0, replace=True,
   random_state=seed)
    pos_test = test[test["y"] == 1].sample(frac=5.0, replace=True,
   random_state=seed)
    return pd.concat([train, pos_train]), pd.concat([test, pos_test])
```

Write down the line number of the bug. Then write the fix.

Sources

- Plain logistic regression at 2.4% prevalence: threshold 0.1 gave precision 0.36, recall 0.42; threshold 0.02 gave recall 0.63, precision 0.04; balanced weights at 0.5 gave recall 0.64, precision 0.05. (verified: Run with scikit-learn 1.9.1 in this repository (make_classification, random_state=0).)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B12-L1 Choose the metric from the decision

B12-L1 exercise 1

Hints. (1) What does a constant prediction score? (2) Compare with 1 - prevalence. (3) 0.976.

Answer. b) Nothing useful yet. Predicting all negatives scores the same. Because: r1) Accuracy is dominated by the 97.6% majority class.

With 2.4% positives, 0.976 is the no-skill baseline. Look at recall, precision and average precision.

Common wrong answers.

- `a`: A constant 'negative' prediction scores 0.976 too.
- `c`: That would be recall. Accuracy mixes both classes.
- `d`: Accuracy says nothing about probability calibration.
- `b|r2`: Right answer, wrong reason. Accuracy works for balanced classes with equal costs.

B12-L1 exercise 2

Hints. (1) A random ranking's precision at every recall level is about the prevalence. (2) What is the prevalence? (3) About 5%.

Answer.

```
1 0.05
```

The no-skill average precision equals the prevalence. Always report it beside the model's score.

Common wrong answers.

- `0.5`: 0.5 is the random baseline for ROC AUC. For average precision the baseline is the prevalence, here 5%.

B12-L1 exercise 3

Hints. (1) np.sort sorts ascending. (2) Count from the end. (3) `-200`, then compare with `threshold`.

Answer.

```
threshold = np.sort(scores)[-200]
flagged = scores >= threshold
```

The threshold comes from capacity, not from 0.5. Ties at the threshold can flag a few more than 200.

Common wrong answers.

- g1 = `200`: np.sort is ascending. The 200th largest value is at index -200.

B12-L1 exercise 4

Hints. (1) flagged = scores >= threshold. tp counts flagged positives. (2) precision = tp / flagged count. recall = tp / positive count. (3) Return None when the denominator is 0.

Answer.

```
import numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score


def evaluate(y_true: np.ndarray, scores: np.ndarray, threshold: float) -> dict[str, float | None]:
    """Ranking metrics, threshold metrics and the prevalence baseline.

    Args:
        y_true: Binary labels.
        scores: Model scores, higher means more likely positive.
        threshold: Flag cases with score >= threshold.

    Returns:
        prevalence, roc_auc, average_precision, precision, recall.
    """
    flagged = scores >= threshold
    tp = int((flagged & (y_true == 1)).sum())
    n_flagged = int(flagged.sum())
    n_pos = int((y_true == 1).sum())
    return {
        "prevalence": float(y_true.mean()),
        "roc_auc": float(roc_auc_score(y_true, scores)),
        "average_precision": float(average_precision_score(y_true, scores)),
        "precision": tp / n_flagged if n_flagged > 0 else None,
        "recall": tp / n_pos if n_pos > 0 else None,
    }
```

Counts make precision and recall transparent. Returning None for an empty denominator avoids a misleading 0.0 and a crash.

B12-L1 exercise 5

Hints. (1) Is capacity fixed? (2) Are costs known and unequal? (3) Is there a threshold yet?

Answer.

- Fraud with fixed review capacity: precision in the top 150, the share of analyst time spent on real fraud.
- Comparing rankers: ROC AUC, with average precision beside it for rare positives.
- Unequal, known costs: total cost at the threshold, which picks the threshold too.

Metrics follow the decision: capacity, costs, or pure ranking.

Common wrong answers.

- fraud = `accuracy`: At 1% prevalence, flagging nothing is 99% accurate.
- credit = `auc`: AUC ignores the threshold and the cost difference, which decide profit.
- compare = `cost`: Without a threshold or agreed costs, compare rankings first.

B12-L2 Calibration, when the probability itself matters

B12-L2 exercise 1

Hints. (1) What does AUC measure? (2) What would a calibrated model show in the 0.8 to 0.9 bin? (3) About 85% observed.

Answer. a) It ranks well but is poorly calibrated. Because: r1) AUC measures ordering only. The probabilities can be far off while the order is right.

Good ranking, poor calibration. Fine for a top-k list, wrong for expected-value sums.

Common wrong answers.

- `b`: Calibrated would mean about 85% churn in that bin, not 20%.
- `c`: Nothing contradicts the AUC. Ranking and calibration are separate properties.
- `d`: 0.85 AUC is good ranking.
- `a|r2`: Right answer, wrong reason. AUC says nothing about calibration.

B12-L2 exercise 2

Hints. (1) Brier = mean of (p - y)^2. (2) For p = 0.25: three rows of 0.0625 and one of 0.5625. (3) For p = 0.5 every row is 0.25.

Answer.

```
0.1875 0.25
```

The prevalence is the best constant prediction. A model worse than 0.1875 here is worse than knowing only the base rate.

Common wrong answers.

- `0.25 0.25`: Predicting the prevalence (0.25) gives (3 × 0.0625 + 0.5625) / 4 = 0.1875.

B12-L2 exercise 3

Hints. (1) The scikit-learn wrapper in sklearn.calibration. (2) Platt scaling's other name. (3) `CalibratedClassifierCV` and `sigmoid`.

Answer.

```
calibrated = CalibratedClassifierCV(model, method="sigmoid",
    cv=5).fit(X_train, y_train)
```

`CalibratedClassifierCV` fits the model and a calibration map on separate folds, so calibration is learned on data the model did not train on.

Common wrong answers.

- g2 = `isotonic`: Isotonic works, but overfits small samples. The prompt asks for sigmoid.

B12-L2 exercise 4

Hints. (1) `edges = np.linspace(0.0, 1.0, n_bins + 1)`. (2) Group a DataFrame of bin, y and p by bin. (3) Named aggregation: mean of p, mean of y, size.

Answer.

```
import numpy as np
import pandas as pd


def calibration_table(y: np.ndarray, p: np.ndarray, n_bins: int) ->
    pd.DataFrame:
    """Mean predicted probability and observed rate per equal-width bin.

    Args:
        y: Binary outcomes.
        p: Predicted probabilities.
```

```
        n_bins: Number of equal-width bins on [0, 1].

    Returns:
        One row per non-empty bin: mean_predicted, observed, n.
    """
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    idx = np.searchsorted(edges[1:-1], p, side="right")
    df = pd.DataFrame({"bin": idx, "y": y, "p": p})
    table = df.groupby("bin").agg(mean_predicted=("p", "mean"),
observed=("y", "mean"), n=("y", "size"))
    return table.reset_index(drop=True)
```

Observed is the positive rate **within** each bin, the number the predicted mean should match. groupby leaves empty bins out.

B12-L2 exercise 5

Hints. (1) Does the use read the order or the number? (2) What does summing probabilities assume? (3) What do balanced weights do to probabilities?

Answer.

- A top-k list needs order, not calibrated values.
- A count forecast sums probabilities, so they must be calibrated.
- Reweighting distorts probabilities. Recalibrate on held-out data and check the Brier score against the constant baseline.

Deliver ranking where order is enough. Deliver calibrated probabilities where the number is used.

Common wrong answers.

- calllist = `calibrated`: Calibration does not change the order, so it does not change who is in the top 500.
- forecast = `ranking`: A sum of probabilities is only right if the probabilities are right.
- training = `balanced_only`: In this lesson's example, balanced weights moved the mean probability from 0.024 to 0.42.

B12-L3 Imbalanced classes and error analysis by segment

B12-L3 exercise 1

Hints. (1) Is the problem the ranking or the cut-off? (2) What does 0.5 mean at 2% prevalence? (3) Choose the threshold from the decision.

Answer. b) Lower the threshold to match review capacity or costs. Because: r1) The model's ranking may be fine. Only the cut-off is wrong for rare positives.

At 2% prevalence few cases score above 0.5. Pick the threshold that matches capacity or cost, and check precision and recall there.

Common wrong answers.

- `a`: Never resample the test set. Its balance must match production.
- `c`: A different model does not fix a threshold chosen by default.
- `d`: Accuracy would look excellent while the model finds nothing.
- `b|r2`: Right answer, wrong reason. A lower threshold trades precision for recall. It does not improve the model.

B12-L3 exercise 2

Hints. (1) Recall is found over positives. (2) Overall pools the counts. (3) 84 / 120.

Answer.

```
email 0.8
chat 0.2
overall 0.7
```

An overall 0.7 hides a segment at 0.2. Pooled rates weight large segments more, so small segments disappear from the headline.

Common wrong answers.

- `email 0.8 chat 0.2 overall 0.5`: Overall recall pools the counts: 84 of 120 is 0.7, not the mean of the two rates.

B12-L3 exercise 3

Hints. (1) What must exist before training? (2) What do segment metrics depend on? (3) Reading examples comes last.

Answer.

1. Split train and test, stratified, test left at its natural balance
2. Train the model normally
3. Choose the threshold from capacity or cost on validation data
4. Break test results down by segment, with counts
5. Read the most confident false positives and false negatives

Split, train, threshold, then look for where and why the model fails.

Common wrong answers.

- Split before training, so the test set never influences the model.
- Segment precision and recall depend on the threshold. Choose it first.

B12-L3 exercise 4

Hints. (1) Loop over `df.groupby(segment, sort=True)`. (2) tp counts rows where y and flagged are both 1, within the segment. (3) Build a list of dicts and return `pd.DataFrame(rows)`.

Answer.

```
1  import pandas as pd
2
3
4  def metrics_by_segment(df: pd.DataFrame, segment: str) -> pd.DataFrame:
5      """Count, positives, precision and recall per segment.
6
7      Args:
8          df: Columns y and flagged (0/1) plus the segment column.
9          segment: Name of the segment column.
10
11     Returns:
12         One row per segment, sorted by segment.
13     """
14     rows = []
15     for name, g in df.groupby(segment, sort=True):
16         tp = int(((g["y"] == 1) & (g["flagged"] == 1)).sum())
17         n_flag = int(g["flagged"].sum())
18         n_pos = int(g["y"].sum())
19         rows.append(
20             {
21                 segment: name,
22                 "n": len(g),
23                 "positives": n_pos,
24                 "precision": tp / n_flag if n_flag > 0 else None,
25                 "recall": tp / n_pos if n_pos > 0 else None,
26             }
27         )
28     return pd.DataFrame(rows)
```

Per-segment counts and rates show where the model fails. Missing values for empty denominators stop a segment with no positives looking like a failure.

B12-L3 exercise 5

Hints. (1) Which set must look like production? (2) What does line 7 do to it? (3) Return test unchanged.

Answer. The bug is on line 7, 8.

Fixed version:

```python
import pandas as pd


def prepare_sets(train: pd.DataFrame, test: pd.DataFrame, seed: int) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Oversample training positives. The test set keeps its natural balance."""
    pos_train = train[train["y"] == 1].sample(frac=5.0, replace=True, random_state=seed)
    return pd.concat([train, pos_train]), test
```

Oversampling the test set inflates positives and duplicates them, so recall is measured on a world that does not exist. Resample training data only.

Common wrong answers.

- Line 6: Oversampling training positives is a legitimate choice. The problem is the other set.